

S-JEPA : explication complète et détaillée

Soft Clustering Anchors for Self-Supervised Speech Representation Learning

Document pédagogique pour la réimplémentation de production

Notes de travail — dépôt `sjepa`

25 juin 2026

Résumé

Ce document explique de bout en bout le papier **S-JEPA** (arXiv :2606.19398) et le codebase de référence fourni (`model.py`, `train.py`, `fit_gmm.py`). Il relie chaque concept mathématique à sa ligne de code, recense les écarts entre le papier et le code de référence (importants pour une réimplémentation fidèle), puis répond à une large série de questions « bêtes mais légitimes » dans une FAQ. Objectif du dépôt : réimplémenter S-JEPA pour la production — **entraînement en Python, inférence en Rust**.

Table des matières

1	Vue d'ensemble en une page	1
2	Intuition : cibles dures contre cibles douces	2
3	La loss : KL entre prédiction et postérieur GMM	2
3.1	Le postérieur d'un GMM diagonal	2
3.2	La fonction de coût	2
4	Architecture détaillée ($\leftrightarrow$ <code>model.py</code>)	3
4.1	Le flux de données	3
4.2	Les composants un par un	3
4.3	Convention de masque : attention au piège	3
5	Les cibles : le GMM en phase 1 et en phase 2	4
5.1	Phase 1 — GMM figé sur MFCC (<code>fit_gmm.py</code>)	4
5.2	Phase 2 — GMM en ligne sur features de l'encodeur EMA	4
6	L'entraînement en deux phases	4
7	Les trois innovations de la phase 2	4
7.1	(a) GMM en ligne	4
7.2	(b) Sélection adaptative de couche par <i>effective rank</i>	5
7.3	(c) Décroissance EMA périodiquement commutée (<i>switched EMA</i>)	5
8	L'algorithme complet (Algorithm 1)	5
9	Infrastructure d'entraînement (orientée production)	5
10	Évaluation et résultats	6
11	L'analyse centrale : l'entropie bimodale	6

12 Écarts entre le papier et le code de référence	6
13 Plan de réimplémentation de production	7
14 FAQ — questions « bêtes » mais légitimes	7
14.1 Concepts généraux	7
14.2 Architecture et masquage	8
14.3 Encodeur EMA et phase 2	9
14.4 Entraînement pratique	9
14.5 Après l’entraînement : usage	10
14.6 Questions « pièges »	11
Références principales	11

1 Vue d’ensemble en une page

Le problème. L’apprentissage auto-supervisé (SSL) de la parole est dominé par une recette unique (HuBERT, WavLM) : on clusterise des features acoustiques *hors-ligne* par k -means, chaque frame reçoit un *identifiant de cluster unique* (cible « dure »), puis on entraîne un encodeur à prédire cet ID aux positions masquées via une cross-entropy. On interrompt ensuite l’entraînement pour re-clusteriser tout le corpus sur des features intermédiaires, et on recommence.

Les deux défauts attaqués.

1. **Assignment dure** : aux frontières de phonèmes, transitions et silences, l’ambiguïté acoustique réelle est écrasée dans une seule catégorie.
2. **Pipeline « stop-restart »** : re-clusteriser le corpus entier entre itérations est coûteux et interrompt l’entraînement.

L’idée de S-JEPA.

- **Cibles douces** : la cible n’est plus un ID unique mais la *distribution postérieure complète d’un GMM* (Gaussian Mixture Model) sur K composantes.
- **Loss = KL divergence** entre la prédiction et le postérieur GMM.
- **GMM en ligne** en phase 2 : le GMM se met à jour en continu, ce qui supprime le re-clustering hors-ligne.
- **Architecture JEPA** : encodeur + prédicteur + tête de cluster, avec un encodeur EMA pour alimenter le GMM en phase 2.

Le résultat. Avec un encodeur de 51.8M de paramètres, S-JEPA est la méthode SSL au plus faible WER sous 90M paramètres (**12.10 %** sur LibriSpeech **test-clean**, greedy CTC; 8.50 % avec LM) et égale HuBERT-Base en reconnaissance d’émotion à 55 % de ses paramètres.

À retenir tout de suite. Le codebase fourni ne contient que le *pré-entraînement*. Il n’y a ni évaluation SUPERB, ni extraction de features, ni inférence. C’est exactement ce que notre projet doit construire (cf. §12 et §13).

2 Intuition : cibles dures contre cibles douces

Imaginons une frame audio située pile à la frontière entre deux sons, par exemple entre le « s » et le « t » de « stop ». Acoustiquement, cette frame est *ambiguë* : elle ressemble à 45 % au cluster « s », à 40 % au cluster « t », et un peu à d’autres.

- **Cible dure (HuBERT)** : on force la frame dans *une* classe, disons « s ». L’information « cette frame est aussi presque un t » est *définitivement perdue*.

- **Cible douce (S-JEPA)** : la cible est le vecteur $[0.45, 0.40, 0.05, \dots]$. Le modèle apprend à reproduire toute la distribution, donc *l’ambiguïté est préservée*.

Le papier mesure ceci directement (§11) : sur les frames de parole réelles, l’entropie de la distribution prédite est *bimodale* — soit très confiante, soit en « égalité à deux » —, et plus d’un tiers des frames portent plus d’un bit d’incertitude. C’est la preuve empirique que les cibles douces capturent une information que les cibles dures jetteraient.

3 La loss : KL entre prédiction et postérieur GMM

3.1 Le postérieur d’un GMM diagonal

Un GMM à K composantes et covariance *diagonale* donne, pour un vecteur de features m , la probabilité a posteriori d’appartenir à la composante k :

$$q_k(m) = \frac{\pi_k \mathcal{N}(m; \mu_k, \sigma_k^2)}{\sum_j \pi_j \mathcal{N}(m; \mu_j, \sigma_j^2)}. \quad (1)$$

En pratique on le calcule en espace log pour la stabilité numérique :

$$q_k(m) = \text{softmax}_k \left(\log \pi_k - \frac{D}{2} \log(2\pi) - \frac{1}{2} \sum_{d=1}^D \left[\log \sigma_{k,d}^2 + \frac{(m_d - \mu_{k,d})^2}{\sigma_{k,d}^2} \right] \right). \quad (2)$$

C’est *exactement* ce qu’implémentent `GMMAssigner.posterior`s (`train.py:63`) et `OnlineGMM.posterior`s (`train.py:111`) : le terme $\sum_d (m_d - \mu_{k,d})^2 / \sigma_{k,d}^2$ est la distance de Mahalanobis (`mahal`), $\sum_d \log \sigma_{k,d}^2$ est `log_det`, puis `logsumexp` normalise. Les variances sont « clampées » à 10^{-6} . Le calcul est découpé en blocs (*chunks*) de 500 pour borner la mémoire.

3.2 La fonction de coût

La loss est la divergence de Kullback–Leibler entre le postérieur GMM q_t (la *cible*, calculée sous `no_grad`) et la sortie softmax du prédicteur p_t , moyennée sur un ensemble de positions S :

$$\mathcal{L} = \frac{1}{|S|} \sum_{t \in S} \text{KL}(q_t \| p_t), \quad p_t = \text{softmax}(g_\omega(\hat{z}_t)). \quad (3)$$

Dans le code (`train.py:833-841`) : `log_pred_mask = log_softmax(logits)`, puis `F.kl_div(log_pred_mask, target_mask, reduction='batchmean')`. Seules les positions masquées contribuent au gradient (`loss = loss_mask`) ; la KL sur positions visibles est calculée *sous no_grad* uniquement pour le logging.

Sens de la KL et ordre des arguments. `torch.nn.functional.kl_div(input, target)` attend `input` en *log*-probabilités et calcule $\sum \text{target} \cdot (\log \text{target} - \text{input})$, soit $\text{KL}(\text{target} \| \exp(\text{input})) = \text{KL}(q \| p)$. C’est bien la « forward KL » du papier : la cible q (GMM) pondère l’erreur. À reproduire à l’identique en Rust pour garantir la parité numérique.

4 Architecture détaillée ($\leftrightarrow$ `model.py`)

L’architecture suit le pattern JEPA avec **trois composants entraînés** — encodeur f_ϕ , prédicteur h_ψ , tête de cluster g_ω — plus un **encodeur EMA** $\bar{f}$ non entraîné (utilisé seulement en phase 2).

4.1 Le flux de données

```
waveform [B,1,T]
  v
ConvFeatureExtractor 7 blocs conv, stride total 320 -> frames 20 ms (50 Hz)
| (model.py:30 kernels (10,5)(3,2)x4(2,2)x2, GELU, GroupNorm fp32 au 1er bloc)
post_extract_proj (512->768) -> Fp32LayerNorm
|
zerote les frames masquées (l'encodeur ne voit RIEN aux positions masquées)
| (model.py:209 x = x * mask.unsqueeze(-1))
ConvPositionalEncoding (kernel 128, 16 groupes, weight-norm) -> encoder LayerNorm
|
N x TransformerLayer (MHSA + FFN, post-norm, LayerNorm fp32)
| z = f_phi(x)
+-----+
|
cluster_head(z) JEPAPredictor : injecte mask_token, ajoute pos_embed,
-> logits "visibles" 1 couche Transformer -> z_hat
(logges, NON supervises) puis cluster_head(z_hat) -> logits "masques"
|
KL( logits masques || posteriors GMM ) <- SEUL signal
```

4.2 Les composants un par un

- **ConvFeatureExtractor** (model.py:30) : 7 conv 1D, kernels/strides identiques à HuBERT, stride total = 320 échantillons $\Rightarrow$ frames de 20 ms à 16 kHz (50 Hz). GroupNorm fp32 au premier bloc, GELU partout.
- **GradMultiply** (model.py:68) : multiplie le gradient du frontend CNN par `feature_grad_mult=0.1` pour empêcher le frontend d'apprendre trop vite (repris de HuBERT).
- **ConvPositionalEncoding** (model.py:54) : encodage positionnel *convolutionnel* (kernel 128, 16 groupes, `weight_norm`) ajouté avant la pile de transformers.
- **TransformerLayer** (model.py:78) : attention multi-têtes + FFN, **post-norm**, LayerNorm forcé en fp32 (Fp32LayerNorm) pour la stabilité en précision mixte.
- **OnlineEncoder** (model.py:169) : assemble tout. Contient aussi `cluster_head` (MLP $768 \rightarrow 768 \rightarrow K$) et `predictor`.
- **JEPAPredictor** (model.py:119) : `mask_token` apprenable injecté aux positions masquées, `pos_embed` (capacité 750 frames = 15 s), **une seule** couche `TransformerEncoderLayer` (8 têtes, FFN = 1536), puis `proj`. Il ne remplit *que* les positions masquées.
- **cluster_head** : MLP partagé entre chemin visible et masqué, mais seul le chemin masqué entre dans la loss.

4.3 Convention de masque : attention au piège

Dans le *code*, `mask` vaut **1 = visible**, **0 = masqué** (train.py:812 : `mask = (~mask_bool).float()`), ce qui est l'*inverse* de la notation du papier. L'encodeur fait `x = x * mask.unsqueeze(-1)` : il met donc à zéro les positions *masquées*. Le `mask_token` n'est injecté que plus tard, par le prédicteur. À fixer comme convention unique et documentée dans notre réimplémentation.

5 Les cibles : le GMM en phase 1 et en phase 2

5.1 Phase 1 — GMM figé sur MFCC (`fit_gmm.py`)

- **Features** : 39-dim MFCC = 13 coefficients cepstraux + Δ + $\Delta\Delta$ (`extract_mfcc`, `fit_gmm.py:254`), frames 20 ms.
- **Reservoir sampling** (`collect_features`, `fit_gmm.py:266`) : échantillonne les frames en flux sans tout charger en mémoire (algorithme de Vitter).
- **Init k -means** (`gmm_init_kmeans`, `fit_gmm.py:313`, 5 itérations) puis **EM** (`fit_gmm_em`, `fit_gmm.py:378`, 20 itérations). Statistiques suffisantes accumulées par chunks ; jamais la matrice $N \times K$ entière.
- **Sortie** : `gmm.pt` = {means, variances, weights, K , dim}. $K = 100$ dans le papier.
- Le GMM est ensuite **figé** pendant toute la phase 1.

5.2 Phase 2 — GMM en ligne sur features de l’encodeur EMA

`OnlineGMM` (`train.py:90`) :

- **Features** : 768-dim, sortie de l’encodeur EMA $\bar{f}$ à la couche active ℓ^* .
- **Mise à jour** (`OnlineGMM.update`, `train.py:134`) : à chaque batch on calcule des moyennes/-variances/poids *pondérés par les responsabilités* $q_{n,k}$, puis une mise à jour EMA :

$$\theta \leftarrow \alpha \theta + (1 - \alpha) \hat{\theta}, \quad \theta \in \{\pi, \mu, \sigma^2\}, \quad \alpha = \text{param_ema} = 0.999. \quad (4)$$

- **Aucun gradient ne traverse le GMM**. Coût $O(B \cdot K \cdot D)$ par pas, intégré dans la passe forward $\Rightarrow$ pas de re-clustering hors-ligne.
- **Seeding** (`train.py:610-737`) : au début de la phase 2, le GMM est initialisé par un mini-batch k -means (Sculley 2010) sur les features de l’encodeur EMA, puis broadcasté à tous les rangs.

6 L’entraînement en deux phases

Les deux phases forment **une seule trajectoire d’optimisation continue**, mais sont lancées comme **deux exécutions** du même `train.py`.

Phase 1 (GMM MFCC figé)	Phase 2 (GMM en ligne)
<code>-gmm_path gmm_mfcc.pt</code> , sans <code>-online_gmm</code>	<code>-online_gmm -reseed_gmm -K 500</code> , repart de <code>-pt_ckpt</code>
Cibles = GMM MFCC figé	Cibles = GMM en ligne sur features EMA
Loss sur masqué + visible	Loss masqué+visible (début) puis masqué seul
Augmentation activée ($p = 0.25$)	Augmentation activée puis désactivée
$K = 100$, LR = 10^{-4}	$K = 500$, LR = 2.5×10^{-5}

7 Les trois innovations de la phase 2

Ce sont les vraies contributions techniques : elles remplacent le stop-restart de HuBERT par un entraînement continu.

7.1 (a) GMM en ligne

Décrit ci-dessus. Supprime la passe de re-labellisation du corpus entier entre itérations.

7.2 (b) Sélection adaptative de couche par *effective rank*

Quelle couche du transformer alimente le GMM ? HuBERT la choisit à la main ; S-JEPA l'automatise via le **rang effectif** :

$$\text{erank}(Z) = \exp\left(-\sum_i p_i \log p_i\right), \quad p_i = \frac{\sigma_i}{\sum_j \sigma_j}, \quad (5)$$

où les σ_i sont les valeurs singulières d'un batch de 2000 embeddings centrés. C'est l'exponentielle de l'entropie de Shannon du spectre : élevé quand beaucoup de valeurs singulières sont comparables (représentation « riche »). Implémentation : `train.py:886-912` (`torch.linalg.svdvals`, normalisation, entropie, lissage EMA, puis `argmax`). Observé : $\ell^* = 2$ en fin de phase 1, puis transition vers $\ell^* = 4$ au milieu de la phase 2.

7.3 (c) Décroissance EMA périodiquement commutée (*switched EMA*)

Tension fondamentale : l'encodeur EMA *alimente* le GMM, dont les composantes sont estimées sur ces mêmes features.

- EMA trop lente $\Rightarrow$ le GMM ne voit jamais les améliorations récentes de l'encodeur.
- EMA trop rapide $\Rightarrow$ la distribution cible bouge à chaque pas.

Solution : alterner entre deux valeurs fixes,

$$\alpha_t \in \{\alpha_{\text{fast}} = 0.999, \alpha_{\text{slow}} = 0.9999\}, \quad \bar{\phi} \leftarrow \alpha_t \bar{\phi} + (1 - \alpha_t) \phi, \quad (6)$$

avec une cadence moyenne d'environ 20 000 pas. C'est l'**analogue continu** du re-cluster hors-ligne : phase rapide = « le GMM rattrape l'encodeur » ; phase lente = « cible stable, l'encodeur apprend contre elle ». Échelles caractéristiques : $\tau(\alpha) = 1/(1 - \alpha)$, soit 1 000 pas (fast) et 10 000 pas (slow).

8 L'algorithme complet (Algorithm 1)

Correspondance directe avec la boucle `train.py:746-960` :

```
Init: f_phi, h_psi, g_omega, encodeur EMA f_bar, GMM, couche ell*, alpha <- alpha_fast
Pour chaque minibatch x :
  x_aug, x_clean <- Augment(x) # train.py:781
  m <- MFCC(x_clean) [Ph.1] ou f_bar(x_clean) a ell* [Ph.2] (no_grad)
  q <- GMM.posterior(m) # :795 / :806
  si Phase 2 :
    GMM <- EMA-update depuis m, q # :860 (online, no grad)
    si pas de verif-couche : ell* <- argmax_l erank(Z^l) # :886-912
  mask <- SampleBlockMask() # :811
  z <- f_phi(x_aug, mask) ; z_hat <- h_psi(z, mask) # :815
  p_t <- softmax(g_omega(z_hat_t)) pour t dans M
  L <- (1/|M|) somme KL(q_t || p_t) ; backward ; step # :837-849
  si Phase 2 : flip alpha ; f_bar <- alpha f_bar + (1-alpha) f_phi # :856-857
Retour: f_phi (predicteur, cluster_head, GMM jetes)
```

Masquage par blocs (`compute_mask_indices`, `train.py:362`) : on tire `num_spans` spans de 10 frames jusqu'à viser un ratio de 0.65. La variante rapide ne garantit pas exactement 0.65 (chevauchements possibles) ; une variante exacte `_slow` existe en `train.py:348`.

9 Infrastructure d'entraînement (orientée production)

- **DeepSpeed** pour le multi-GPU (8 GPU, batch global 192), config `ds_config_jepa_pretrain.json`.
- **Données** : manifests JSONL `{"wav_path": ...}`, lus en streaming (`StreamingDataset`, `train.py:243`). Un `path_index.txt` globalement mélangé est construit une seule fois sur rank 0 ; chaque worker lit des slices disjointes (`i % total == wid`).

- **Checkpoint portable** (`portable_step*.pt`, `train.py:917-942`) : bundle {encodeur online, encodeur EMA, params GMM, step}, agnostique du framework, écrit dans un thread daemon pour ne pas bloquer l’entraînement.
- **Augmentation** (`DenoiseAugmentor`, `train.py:190`) : noise mixing au SNR + utterance mixing, suivant WavLM. Le GMM voit le signal *propre*, l’encodeur voit le signal *augmenté*.

10 Évaluation et résultats

Protocole **SUPERB** : encodeur *gelé*, on entraîne de petites têtes par tâche.

Tâche	S-JEPA (51.8M)	Comparaison
ASR (WER test-clean)	12.10 % greedy ; 8.50 % +LM	meilleur SSL <90M (DeCoAR 2.0 : 13.02 % à 89.8M)
Émotion (ER, acc.)	64.83 %	$\approx$ HuBERT-Base (64.92 %) à 55 % des params
Slot Filling (F1/CER)	83.05 / 33.17	$\approx$ DeCoAR 2.0

Plus du **probing** (sondes linéaires/MLP sur représentations gelées) : S-JEPA domine Speaker ID, Gender, Chapter ID, et gagne en Phoneme avec une sonde MLP. Conclusion : nouvelle **frontière de Pareto sous 90M paramètres**, sans *teacher* ni re-clustering hors-ligne.

11 L’analyse centrale : l’entropie bimodale

Le résultat empirique qui justifie les cibles douces : sur $\sim 153\,000$ frames held-out, l’entropie par frame de la prédiction

$$H_t = - \sum_{k=1}^K p_{t,k} \log_2 p_{t,k} \in [0, \log_2 K] \quad (7)$$

est **bimodale** :

- un mode confiant < 0.3 bit (régions phonétiquement stables) ;
- un mode secondaire ≈ 1 bit = égalité parfaite à deux clusters (frontières acoustiques) ;
- **36.6 %** des frames portent plus de 1 bit.

Lecture en bits : $H = 1$ bit = égalité 50/50 ; $H = 2$ bits = uniforme à quatre ; $H = \log_2 500 \approx 9$ bits = uniforme sur tous les clusters (jamais atteint). La bimodalité est l’observation « porteuse » : ce n’est ni du flou diffus (qui serait unimodal), ni un repli dégénéré vers l’uniforme (qui s’empilerait à $\log_2 K$).

12 Écarts entre le papier et le code de référence

Cruciaux pour une réimplémentation *fidèle au papier*.

Élément	Papier	Code fourni
Couches encodeur	6 (51.8M)	default 12 ; <code>-num_layers 6</code> possible
Switched EMA ($\alpha_{\text{fast}}/\alpha_{\text{slow}}$)	spécifié en détail	absent (<code>-ema_decay</code> fixe)
Transition loss vis+masq $\rightarrow$ masqué seul	décrite (annexe)	code n’a que masqué (<code>loss = loss_mask</code>)
Eval SUPERB / inférence	résultats reportés	non fourni
K phase 1 / phase 2	100 / 500	configurable (<code>-K</code>)

13 Plan de réimplémentation de production

Décision arrêtée : *entraînement en Python, inférence en Rust.*

- **Python (entraînement + recherche)** : package modulaire reprenant `fit_gmm.py`, `train.py`, `model.py`, enrichi des éléments manquants (6 couches, switched EMA, transition de loss), testé, avec configs et reprise propre. Production des checkpoints portables.
- **Rust (inférence + serving)** : moteur d'inférence haute-performance chargeant le checkpoint portable (encodeur seul — le prédicteur, la tête de cluster et le GMM sont jetés après pré-entraînement). Pistes : `tch` (bindings libtorch) ou `candle`. Tests de **parité numérique** contre la sortie Python.
- **Évaluation** : à construire (extraction de features gelées + têtes SUPERB / probing). Peut rester en Python dans un premier temps.

14 FAQ — questions « bêtes » mais légitimes

14.1 Concepts généraux

Q. C'est quoi « auto-supervisé » (SSL) au juste ?

R. On entraîne le modèle *sans étiquettes humaines*. Le signal d'apprentissage est fabriqué à partir des données elles-mêmes : ici, on masque des morceaux du signal et on demande au modèle de prédire ce qui manque (dans un espace de features, pas le son brut). Aucune transcription n'est nécessaire pour le pré-entraînement.

Q. Que signifie « JEPA » ?

R. *Joint-Embedding Predictive Architecture*. On ne prédit pas les pixels ou les échantillons audio bruts, mais une *représentation* (embedding) de la partie masquée. Un encodeur encode le contexte visible, un prédicteur « remplit » les positions masquées dans l'espace latent.

Q. Pourquoi prédire dans l'espace latent plutôt que reconstruire le son ?

R. Reconstruire le signal brut force le modèle à modéliser des détails non pertinents (bruit de fond, phase exacte). Prédire une représentation laisse le modèle se concentrer sur la structure utile (contenu phonétique, prosodie).

Q. C'est quoi un GMM, expliqué simplement ?

R. Un *Gaussian Mixture Model* suppose que les données sont générées par un mélange de K « bulles » gaussiennes. Chaque bulle a un centre (μ_k), une largeur (σ_k^2) et un poids (π_k). Le *postérieur* donne, pour un point, la probabilité d'appartenir à chaque bulle. C'est un k -means « mou » qui rend des probabilités au lieu d'un seul gagnant.

Q. Quelle différence concrète entre k -means et GMM ici ?

R. k -means assigne chaque point à *un* cluster (cible dure). Le GMM rend une *distribution* sur tous les clusters (cible douce). C'est exactement le cœur de S-JEPA : remplacer la cible dure de HuBERT par la cible douce du GMM.

Q. Pourquoi « covariance diagonale » et pas complète ?

R. Une covariance complète a D^2 paramètres par composante (ici 768^2), trop coûteux et instable. Diagonale = on suppose les dimensions indépendantes, D paramètres par composante. Standard pour la parole, et le postérieur a une forme close simple (Eq. 2).

Q. C'est quoi la divergence KL, en une phrase ?

R. Une mesure d'écart entre deux distributions de probabilité. $KL(q \parallel p) = 0$ si et seulement si $q = p$, et grandit quand p s'éloigne de la cible q . Minimiser la KL, c'est rendre la prédiction p aussi proche que possible de la cible q .

14.2 Architecture et masquage

Q. Pourquoi des frames de 20 ms ?

R. Le frontend CNN a un stride total de 320 échantillons. À 16 kHz, $320/16000 = 20$ ms par frame, soit un débit de 50 frames/seconde. C'est le standard HuBERT/wav2vec et un bon compromis pour la parole.

Q. Pourquoi mettre les frames masquées à zéro *avant* le transformer, mais injecter le mask token *après* (dans le prédicteur) ?

R. L'encodeur doit produire une représentation du *contexte visible* seulement. Si on lui donnait déjà un mask token, il « tricherait ». Le prédicteur, lui, reçoit le contexte encodé et doit *deviner* le contenu masqué : c'est là qu'on place le mask token apprenable.

Q. Pourquoi multiplier le gradient du CNN par 0.1 ?

R. Le frontend convolutionnel apprend très vite et peut déstabiliser l'entraînement. Réduire son gradient ($\gamma_{\text{feat}} = 0.1$) le fait évoluer plus lentement que les transformers. Astuce héritée de HuBERT.

Q. Pourquoi un encodage positionnel *convolutionnel* plutôt que sinusoïdal ?

R. Pour la parole, une convolution capture les positions *relatives* de façon souple et performe mieux empiriquement que les positions absolues sinusoïdales. C'est le choix de wav2vec 2.0 / HuBERT.

Q. Le prédicteur n'a qu'une couche transformer, ce n'est pas trop peu ?

R. C'est volontaire : le prédicteur est un module *léger et jetable*. Tout le travail représentationnel doit être fait par l'encodeur, qui est le seul composant conservé après pré-entraînement.

Q. Pourquoi 65 % de masquage, c'est énorme, non ?

R. Oui, et c'est voulu : un masquage élevé rend la tâche difficile et force l'encodeur à apprendre une vraie structure plutôt qu'une interpolation locale. 65 % avec des spans de 10 frames est la valeur HuBERT.

14.3 Encodeur EMA et phase 2

Q. C'est quoi l'encodeur EMA et pourquoi en a-t-on besoin ?

R. C'est une copie « lente » de l'encodeur, dont les poids sont une moyenne mobile exponentielle de l'encodeur entraîné. En phase 2, il fournit des features *stables* sur lesquelles le GMM est ajusté. Sans cela, le GMM poursuivrait une cible qui bouge à chaque pas (instable).

Q. Si le GMM dépend de l'encodeur et que l'encodeur apprend contre le GMM, n'est-ce pas circulaire / risque d'effondrement ?

R. C'est exactement la tension que gère la *switched EMA* : en alternant une phase « rapide » (le GMM rattrape l'encodeur) et une phase « lente » (cible figée, l'encodeur apprend contre elle), on évite que les deux dérivent ensemble vers une solution dégénérée. L'EMA et le `no_grad` sur le GMM brisent la boucle de rétroaction directe.

Q. Pourquoi le GMM ne reçoit-il aucun gradient ?

R. Le GMM est une *cible*, pas un paramètre à optimiser par descente de gradient. Ses paramètres bougent par mise à jour EMA des statistiques suffisantes (moyennes/variances pondérées). Laisser un gradient le traverser créerait un raccourci trivial vers l'effondrement.

Q. C'est quoi le *rang effectif* et pourquoi ça aide à choisir une couche ?

R. C'est une mesure de « richesse » d'une représentation : si les directions principales (valeurs singulières) sont nombreuses et équilibrées, le rang effectif est élevé (l'information est étalée sur beaucoup de dimensions). Une couche à haut rang effectif tend à mieux séparer les sons, d'où son choix comme entrée du GMM — sans avoir besoin d'étiquettes.

Q. Pourquoi $K = 100$ puis $K = 500$?

R. Pour calquer les cardinalités des itérations 1 et 2 de HuBERT. En phase 1, les features MFCC sont grossières (100 clusters suffisent) ; en phase 2, les features de l'encodeur sont plus fines et supportent un vocabulaire plus large (500 clusters), proche du sous-phonémique.

14.4 Entraînement pratique

Q. Combien de données et de calcul faut-il ?

R. Le papier pré-entraîne sur $\sim 83\,000$ heures (LibriLight + sous-ensemble anglais de Granary), sur 8 GPU, batch global 192, séquences jusqu'à 15 s. C'est un entraînement « gros corpus ». Pour expérimenter, on peut commencer sur LibriSpeech (960 h) à échelle réduite.

Q. Pourquoi deux *lancements* séparés et pas une seule commande ?

R. Par simplicité d'ingénierie : la phase 2 a besoin du checkpoint de la phase 1 (`-pt_ckpt`) et active des options différentes (`-online_gmm`, K , LR). Conceptuellement c'est *une* trajectoire continue ; en pratique c'est deux exécutions enchaînées.

Q. Que contient le « checkpoint portable » et pourquoi l'appeler ainsi ?

R. Il bundle l'encodeur online, l'encodeur EMA, les paramètres du GMM et le numéro de step, dans un simple `.pt` indépendant de DeepSpeed. « Portable » = on peut le recharger sans l'infrastructure d'entraînement, pour reprendre ou pour extraire des features. C'est lui qu'on chargera côté Rust pour l'inférence (en ne gardant que l'encodeur).

Q. Pourquoi désactiver l'augmentation plus tard en phase 2 ?

R. Parce que le GMM calcule sa cible sur le signal *propre* alors que l'encodeur voit le signal *augmenté*. Cela impose une contrainte d'invariance supplémentaire qui, à ce stade, rend la prédiction masquée plus difficile que nécessaire. Couper l'augmentation aligne ce que voient les deux côtés.

Q. Pourquoi la loss sur positions *visibles* est-elle calculée mais pas optimisée ?

R. Prédire un cluster là où l'encodeur voit déjà l'entrée est trop facile et apporte des gradients largement redondants. On la garde pour le *monitoring* (logging) mais on n'optimise que les positions masquées, comme data2vec / wav2vec 2.0 / HuBERT.

14.5 Après l'entraînement : usage

Q. Une fois entraîné, qu'est-ce qu'on garde ?

R. Uniquement l'encodeur f_ϕ . Le prédicteur, la tête de cluster et le GMM sont jetés. C'est l'encodeur (51.8M params) qui produit les représentations utilisées en aval.

Q. Comment utilise-t-on S-JEPA pour une tâche concrète (ASR, émotion) ?

R. On *gèle* l'encodeur et on entraîne une petite tête spécifique à la tâche sur ses features (protocole SUPERB). L'encodeur n'est pas fine-tuné ; seule la tête apprend. C'est rapide et révèle la qualité des représentations.

Q. Pourquoi l'inférence en Rust et pas tout en Python ?

R. Pour la production : Rust offre des latences prévisibles, pas de GIL, une empreinte mémoire maîtrisée et un déploiement en binaire autonome. L'entraînement reste en Python (écosystème PyTorch/recherche), l'inférence critique passe en Rust. On vérifie la parité numérique entre les deux.

Q. Comment garantir que le Rust donne les mêmes résultats que le Python ?

R. Par des *tests de parité* : on sauvegarde des entrées/sorties de référence depuis Python (frontend CNN, chaque couche transformer, sortie finale) et on vérifie que l'implémentation Rust reproduit ces tenseurs à une tolérance numérique près. Points sensibles : LayerNorm en fp32, ordre des opérations d'attention, weight-norm de l'encodage positionnel.

Q. Le GMM est-il nécessaire à l'inférence ?

R. Non. Le GMM ne sert qu'à fabriquer les *cibles* pendant le pré-entraînement. À l'inférence on ne fait tourner que l'encodeur ; il n'y a ni GMM, ni prédicteur, ni masquage.

Q. Et le masquage, à l'inférence ?

R. Désactivé. On donne le signal complet à l'encodeur et on lit ses features frame par frame. Le masquage est un artefact d'entraînement.

14.6 Questions « pièges »

Q. Si on jette le GMM et le prédicteur, à quoi servaient-ils ?

R. Ils ne servent qu'à *façonner* l'encodeur pendant l'entraînement. Le GMM fournit des cibles douces, le prédicteur force l'encodeur à produire un contexte assez riche pour qu'on puisse deviner les frames masquées. Une fois l'encodeur bon, les échafaudages tombent.

Q. S-JEPA est-il « juste HuBERT avec un GMM » ?

R. En esprit oui (« HuBERT à cibles douces dans une architecture JEPA »), mais trois différences techniques comptent : (1) cibles *douces* GMM au lieu de dures *k*-means, (2) GMM *en ligne* au lieu de re-clustering hors-ligne, (3) couche source du GMM choisie *automatiquement* par rang effectif. C'est la *recette* combinée qui est la contribution, pas un nouveau bloc d'architecture.

Q. Pourquoi S-JEPA bat-il des modèles plus gros sur certaines tâches ?

R. L'hypothèse du papier : les cibles douces transportent de l'information que les cibles dures jettent (la masse inter-clusters aux frontières). Sur 36.6 % des frames, cette information existe. Le papier ne *prouve* pas la causalité, mais corrèle cette structure d'incertitude avec l'efficacité paramétrique.

Q. La « switched EMA » est-elle indispensable ?

R. Le papier la présente comme importante mais reconnaît (Limitations) qu'elle manque d'une comparaison contrôlée. Le code de référence ne l'implémente même pas (EMA fixe). Pour la prod, on l'implémentera proprement *et* on gardera l'option EMA fixe, afin de pouvoir comparer.

Q. Tout ceci marche-t-il sur d'autres langues / de l'audio non-vocal ?

R. Non testé : le papier est entièrement en anglais et le signale comme limitation. Langues tonales, faibles ressources et audio non-vocal sont des extensions ouvertes.

Références principales

- Papier S-JEPA : `papers/sources/arXiv-2606.19398v1/main.tex`.
- Code de référence : `papers/sources/s-jepa-main/{model,train,fit_gmm}.py`.
- Antécédents : HuBERT (cibles dures, deux itérations), wav2vec 2.0 (masquage, encodage positionnel convolutionnel), data2vec / BYOL (encodeur EMA), I-JEPA (pattern encodeur-prédicteur).